

Lesson 6

Part 1

This lesson demonstrates a **Denial of Service (DoS)** vulnerability in the DVSA application. The main affected components are **Amazon API Gateway and AWS Lambda**, which handle incoming API requests. By sending a large number of concurrent authenticated requests to the `/dvsa/order` endpoint, the backend becomes overwhelmed and fails to process requests correctly. The security impact is a loss of **availability**, where legitimate users cannot access or use the system reliably. The main weakness is the lack of effective **rate limiting and request throttling**, allowing excessive requests to consume backend resources.

Part 2

This vulnerability exists because the application does not properly control the number of incoming requests. The API endpoint accepts multiple concurrent requests without sufficient **rate limiting or abuse protection**, allowing a single client to overload the system. As a result, backend services such as AWS Lambda become unstable or fail under high load, leading to errors such as HTTP 500 and 502. The root cause is weak **resource management and request-handling design**, where the system does not enforce fair usage or protect against excessive traffic.

Part 3

The DVSA application was deployed on AWS using a serverless architecture. The frontend is hosted at:

<http://dvsa-website-test-382764425967-us-east-1.s3-website.us-east-1.amazonaws.com/>

The backend API endpoint used in this test is:

<https://a2b59e5rj4.execute-api.us-east-1.amazonaws.com/dvsa/order>

The request method used was POST, with a JSON body:

```
{"action":"inbox"}
```

The test was performed using an authenticated user with a valid JWT token obtained from the DVSA login system (Amazon Cognito).

The following AWS services were involved:

- Amazon API Gateway (request entry point)
- AWS Lambda (backend processing)
- Amazon CloudWatch (logging and monitoring)

The tools used for testing were:

- Browser Developer Tools (to capture the API request)
- curl (to send HTTP requests)
- Linux terminal

Part 4

- 1- Open the DVSA website and log in with a valid user account.
- 2- Open Developer Tools → Network tab and perform an action to capture the API request.
- 3- Identify the backend endpoint:
`https://<api-id>.execute-api.<region>.amazonaws.com/dvsa/order`
- 4- Copy the request as cURL and test a single request
- 5- Send multiple concurrent requests to simulate DoS
- 6- Observe that many requests return **500 and 502 errors**, indicating the system fails under load.

Part 5

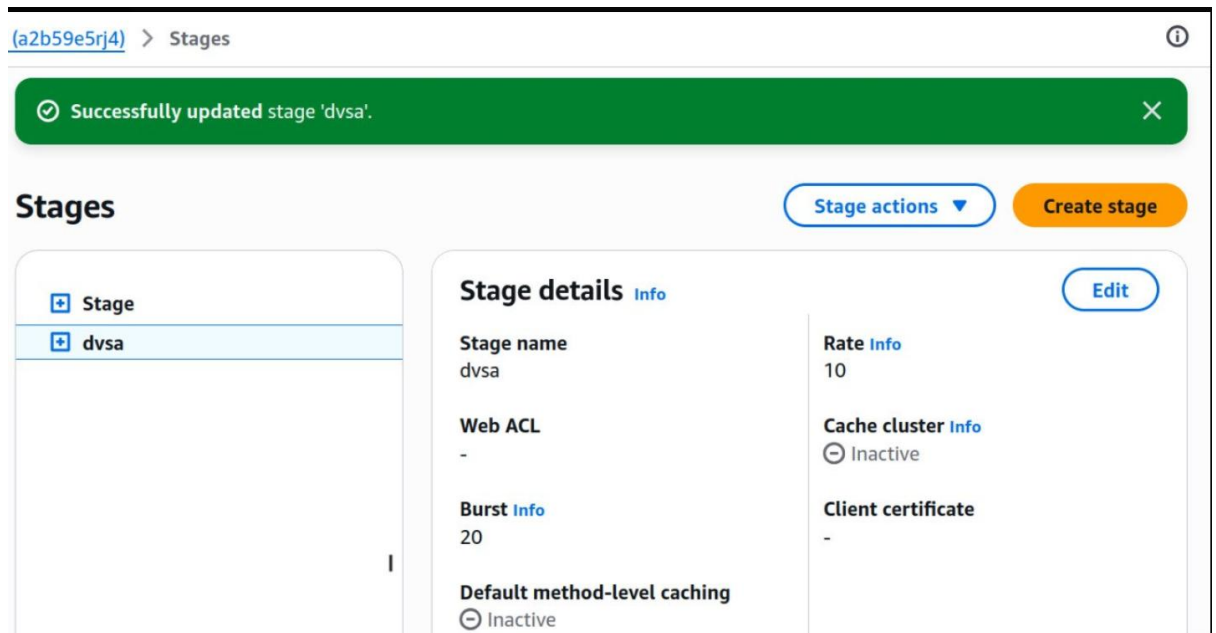
```
-X POST \
-H "authorization: $TOKEN" \
-H 'content-type: application/json' \
--data-raw '{"action":"inbox"}'
bash: https://a2b59e5rj4.execute-api.us-east-1.amazonaws.com/dvsa/order: No such file or directory
faissal@faissal:~$ seq 1 100 | xargs -n1 -P20 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
'https://a2b59e5rj4.execute-api.us-east-1.amazonaws.com/dvsa/order' \
-X POST \
-H "authorization: $TOKEN" \
-H 'content-type: application/json' \
--data-raw '{"action":"inbox"}' | sort | uniq -c
xargs: warning: options --max-args and --replace/-I/-i are mutually exclusive, ignoring previous --max-args value
    6 500
   94 502
```

The vulnerability was confirmed by sending multiple concurrent requests to the API endpoint. The terminal output is in the screenshot. The presence of **HTTP 500 and 502 errors** indicates that the backend system became unstable and failed to process requests under load. These results demonstrate that the system cannot handle a high volume of concurrent requests, confirming a denial-of-service condition affecting availability.

Part 6

The vulnerability can be mitigated at the API Gateway layer by enabling rate limiting and request throttling. The main improvement is to restrict the number of requests that a client can send within a short period of time. This prevents excessive concurrent requests from reaching the backend. By limiting incoming traffic, the system avoids resource exhaustion in AWS Lambda and maintains availability for legitimate users. This mitigation directly addresses the root cause, which is the lack of control over incoming request volume.

Part 7



The fix was implemented by modifying the **API Gateway stage configuration**.

The following changes were applied:

- Throttling enabled
- **Rate limit:** 10 requests per second
- **Burst limit:** 20 requests

These settings were configured in:

- AWS API Gateway → Stages → dvsa → Throttling settings

This change ensures that excessive requests are limited before reaching the backend Lambda function, reducing system overload.

Part 8

```
faisal@faisal:~$ seq 1 100 | xargs -P20 -I{} curl -s -o /dev/null -w "%{http_code}\n" "https://a2b59e5rj4.execute-api.us-east-1.amazonaws.com/dvsa/order"
-X POST -H "authorization: $TOKEN" -H "content-type: application/json" --data-raw '{"action":"inbox"}' | sort | uniq -c
  6 429
  5 500
 89 502
faisal@faisal:~$
```

After applying the fix, the same concurrent request test was repeated. The results is in the screenshot. The presence of HTTP 429 responses confirms that throttling is now active and blocking excessive requests. Although some backend errors (500 and 502) still occur, the system now rejects part of the attack traffic before it reaches the backend. This demonstrates that the mitigation has improved system protection by limiting request rates, while still allowing legitimate requests to be processed under normal conditions.

Part 9

1)Intended Logic and Security Rule(s)

The intended logic of the DVSA system is that user requests are sent through API Gateway to AWS Lambda, where they are processed and a valid response is returned. The system should handle requests efficiently without failure.

The main security rules are:

- The system must limit excessive requests from a single user.
- Backend services must remain stable under normal and moderate load.
- The application should prevent resource exhaustion to ensure availability.

2) Evidence Sources and Behavior Trace

Evidence was collected using:

- Browser Developer Tools (Network tab)
- Terminal output using curl
- AWS CloudWatch logs

During testing:

- A single request returned HTTP 502
- Multiple concurrent requests were sent using seq and xargs
- The results showed:
6 429
5 500
89 502

This shows that under concurrent load, the system produced a large number of error responses and failed to handle requests properly.

3) Deviation Analysis and Classification

The system behavior deviates from the intended logic because it fails to handle multiple concurrent requests and becomes unstable.

Instead of processing requests normally:

- The backend returned **500 and 502 errors**
- The system became unreliable and unavailable

This is classified as:

Intentional misuse / security-relevant abuse

Because:

- An attacker can deliberately send many requests
- This overwhelms the system
- And disrupts normal service (DoS attack)

4) Explainable Fix and Post-Fix Validation

The root issue is the lack of request control at the API level.

The fix was applied by enabling **API Gateway throttling**:

- Rate limit: 10 requests/sec
- Burst limit: 20 requests

This fix works by:

- Limiting how many requests reach the backend
- Preventing resource exhaustion
- Protecting system availability

After applying the fix:

- Some requests returned **429 (Too Many Requests)**
 - This confirms that excessive requests are now blocked
- Although some backend errors (500/502) still exist, the system now partially enforces request limits and improves stability.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Denial of Service (DoS)	The system should limit request rate and remain available under load	Browser DevTools, curl output, CloudWatch logs	System processes requests normally without errors	Multiple concurrent requests caused 500 and 502 errors, showing system failure

Vulnerability	Why This is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification	Optional Latency Before/ After
Denial of Service (DoS)	System failed to handle multiple concurrent requests and became unavailable	Intentional misuse / security-relevant abuse	API Gateway throttling (Rate = 10, Burst = 20)	Presence of 429 responses confirms requests are now limited	Not measured

Part 10

This lesson shows that the system assumed incoming requests would remain within normal limits, without considering abusive or excessive traffic. In a serverless environment, this is important because services like API Gateway and Lambda can be easily overwhelmed if proper controls are not in place. The key lesson learned is the importance of implementing rate limiting and resource protection as part of secure design. Applying principles such as defense in depth and least privilege for resource usage helps prevent similar availability issues in the future.

Lesson 8

Part 1

This lesson demonstrates a **logic vulnerability (race condition)** in the order-processing workflow of the DVSA application. The issue occurs in the backend **AWS Lambda function** that handles billing and order updates through **API Gateway**, with data stored in **DynamoDB**. The vulnerability allows an attacker to manipulate the order quantity after initiating payment, resulting in receiving more items than actually paid for. The security impact is a **financial loss and inconsistent application state**, caused by a lack of proper synchronization and workflow validation between billing and update operations.

Part 2

This vulnerability exists because the application does not enforce proper **workflow sequencing and state validation**. The system allows order updates to be processed even after a billing request has started. There is no mechanism to lock the order, validate its state, or ensure atomic execution of operations. As a result, concurrent requests can modify the order during payment processing, leading to inconsistent data. The root cause is a **business logic flaw** due to missing synchronization and lack of transaction control.

Part 3

The vulnerability was tested using the deployed DVSA application hosted at:

- **DVSA URL:**

<http://dvsa-website-test-382764425967-us-east-1.s3-website.us-east-1.amazonaws.com/>

- **AWS Services involved:**

- Amazon API Gateway (handles incoming requests)
- AWS Lambda (processes billing and order updates)
- Amazon DynamoDB (stores order data in DVSA-ORDERS-DB)

- **Tools used:**

- Browser DevTools (to inspect network requests)
- curl (to send concurrent API requests)
- AWS Console (to verify data in DynamoDB)

- **Test context:**

- A normal user account was used
- An order was created with quantity = 1 before exploitation

Only the above components were used to reproduce and verify the vulnerability.

Part 4

- 1- Open the DVSA website and log in using a valid user account.
- 2- Add a product to the cart with Quantity = **1** and proceed to checkout but do not finalize yet.
- 3- Open Browser DevTools → Network tab and perform a normal checkout once to capture billing request and order update request.
- 4- Copy both requests using “Copy as cURL”
- 5- Prepare two requests: update request and billing request.
- 6- Both requests were executed concurrently using background execution and curl billing_request & sleep 0.05:

```
curl update_request &  
curl billing_request &  
sleep 0.05  
curl update_request &
```
- 7- Verify results by navigating to AWS Console → DynamoDB → DVSA-ORDERS-DB
- 8- Open the latest order record and Inspect the itemList and totalAmount fields
- 9- Observe the Inconsistent State
- 10- The final order shows: Item quantity = **10** and Total amount = **33**

Part 5

```
walt
[1] 7982
[2] 7966
{"status":"ok","msg":"cart updated"}{"status":"ok","amount":33,"token":"zbcikjEggaVo","missing":{}}[1]- Done curl -s "$API" -H "content-type: application/json"
{"status":"ok","msg":"cart updated"}{"status":"ok","amount":33,"token":"zbcikjEggaVo","missing":{}}[1]- Done curl -s "$API" -H "content-type: application/json"
authorization: $TOKEN --data-raw '{"action":"billing","order-id":"$ORDER_ID","data":{"ccn":"4242424242424242","exp":"11/30","cvv":"123"}}'
[2]+ Done curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"update","order-id":"$ORDER_ID","items":{"103
}}'
faisal@faisal:~$
```

Attribute name	Value	Type	Remove
orderId - Partition key	a9969eb7-5a1e-4f9f-8714-6962b65a60e9	String	
userId - Sort key	a468b478-f001-70c2-b028-b26664637ed0	String	
address	Insert a field	Map	Remove
confirmationToken	zbcikjEggaVo	String	Remove
itemList	Insert a field	Map	Remove
1013	10	Number	Remove
orderStatus	200	Number	Remove
paymentTS	1776165928	Number	Remove
totalAmount	33	Number	Remove

[Cancel](#) [Save](#) [Save and close](#)

In the first screenshot, two concurrent API requests were executed:

- A **billing request** to process payment
- An **update request** to modify the order quantity

The responses confirmed that both operations were accepted:

```
{"status":"ok","msg":"cart updated"}
{"status":"ok","amount":33,"token":"..."}
```

This indicates that:

- The system processed both requests successfully
- No validation or restriction prevented concurrent execution

The DynamoDB screenshot shows:

- The expanded itemList containing quantity = 10
- The totalAmount field remaining at 33

This visual evidence confirms that the system stored an inconsistent order state.

Part 6

To mitigate this logic vulnerability, the fix must be applied in the backend AWS Lambda function responsible for handling billing and order updates, as well as in the workflow validation logic of the application.

The main issue is that the system allows order updates during the billing process without enforcing any state validation or synchronization. To address this, the application must implement strict workflow control and state-based validation.

First, the system should enforce that once a billing request begins, the order is marked as locked or in-progress, preventing any further modifications. This can be done by introducing an order status field (e.g., PENDING, PROCESSING, PAID) and validating this status before allowing updates.

Second, the application should ensure atomic operations when processing billing and updates. This can be achieved using conditional updates in DynamoDB or transactional APIs to guarantee that order updates only occur when the order is in a valid state.

Third, input validation logic should be strengthened to reject any update request if the order is already being processed or has been completed.

Part 7

The fix was applied in the backend AWS Lambda functions responsible for updating an order and processing payment. The modified files were:

- DVSA-ORDER-UPDATE / update_order.py
- DVSA-ORDER-BILLING / order_billing.py

The main goal of the fix was to prevent the order from being modified while billing was in progress. To achieve this, I added server-side state validation and conditional DynamoDB updates so that updates are only allowed when the order is still in the 100 (open) state. During billing, the order is immediately moved to 200 (processing), which blocks concurrent update requests.

File 1: update_order.py

Before fix

The update function allowed the cart to be modified without enforcing a strict workflow rule. It only checked whether the order was already paid, which still allowed the cart to be changed while billing was starting.

```
if response["Item"]["orderStatus"] > 110:
```

```
    res = {"status": "err", "msg": "order already paid"}
```

```
    return res
```

```
response = table.update_item(  
    Key={"orderId": orderId, "userId": userId},  
    UpdateExpression=update_expr,  
    ExpressionAttributeValues={  
        ':itemList': itemList  
    }  
)
```

After fix

I changed the logic so that updates are only allowed when the order is exactly 100 (open). I also added a DynamoDB ConditionExpression to make the database reject the update if the order status has changed.

```

if response["Item"]["orderStatus"] != 100:

    res = {"status": "err", "msg": "Order cannot be modified after billing has
started"}

    return res

try:

    response = table.update_item(

        Key={"orderId": orderId, "userId": userId},

        UpdateExpression=update_expr,

        ConditionExpression="orderStatus = :open",

        ExpressionAttributeValues={

            ':itemList': itemList,

            ':open': 100

        }

    )

except ClientError as e:

    if e.response["Error"]["Code"] == "ConditionalCheckFailedException":

        res = {"status": "err", "msg": "Order is no longer editable"}

        return res

    raise

```

Security improvement

This change prevents order updates after billing has begun. The important restriction added here is:

- updates are allowed only when orderStatus = 100
- DynamoDB itself enforces this rule, even under concurrent requests

File 2: order_billing.py

Before fix

The billing function retrieved the order, calculated the total, called the payment API, and only then updated the order status. This left a race window where another request could still modify the cart before payment finished.

```
status = int(json.dumps(response["Item"]['orderStatus'],
cls=DecimalEncoder))
```

```
if status < 120:
```

```
...
```

```
req = http.request("POST", url, body=data, ...)
```

```
...
```

```
response = table.update_item(
    Key=key,
    UpdateExpression=update_expression,
    ExpressionAttributeValues=expression_attributes
)
```

After fix

I added an immediate order lock before billing continues. The billing function now changes the order status from 100 (open) to 200 (processing) as soon as billing starts. I also added a conditional check when writing the final paid state.

```
if status != 100:
```

```
    return {"status": "err", "msg": "order is not open for billing"}
```

```
try:
```

```

table.update_item(
    Key={
        "orderId": orderId,
        "userId": userId
    },
    UpdateExpression="SET orderStatus = :processing",
    ConditionExpression="orderStatus = :open",
    ExpressionAttributeValues={
        ":processing": 200,
        ":open": 100
    }
)
except ClientError as e:
    if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
        return {"status": "err", "msg": "order already locked"}
    raise

```

Later, when payment succeeds, the final order write is protected with another conditional check:

```

table.update_item(
    Key=key,
    UpdateExpression=update_expression,
    ConditionExpression="orderStatus = :processing",
    ExpressionAttributeValues=expression_attributes
)

```

Security improvement

This change adds the missing workflow protection:

- billing can begin only if the order is still 100 (open)
- billing immediately locks the order by setting orderStatus = 200
- the final payment update is allowed only if the order is still in 200 (processing)

Part 8

The screenshot is divided into two main sections. The top section is a terminal window with a dark background, showing a series of commands and their outputs. It starts with a 'wait' command, followed by two 'curl' commands that simulate concurrent 'billing' and 'update' requests. The bottom section is a screenshot of the AWS Management Console, specifically the 'Edit item' page for a DynamoDB table named 'DVSA-ORDERS-DB'. The 'Attributes' table shows the state of the order after the concurrent updates. The 'orderStatus' is 210, and the 'totalAmount' is 330. The 'itemList' contains one item with 'id' 1013 and 'value' 10. The 'confirmationToken' is 'dGZoQNbrqR1'. The 'address' attribute is present but has no value. The 'userId' is 'a468b478-f001-70c2-b028-b26664637ed0'. The 'paymentTS' is '1776179114'. The 'Remove' buttons for each attribute are visible, and the 'Save and close' button is at the bottom right.

```
wait
[1] 14138
[2] 14134
{"status":"ok","msg":"cart updated"}{"status":"ok","amount":330,"token":"dGZoQNbrqR1","missing":{}}[1]- Done
{"status":"ok","msg":"cart updated"}{"status":"ok","amount":330,"token":"dGZoQNbrqR1","missing":{}}[2]- Done
curl -s -X POST -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"billing","order-id":"$ORDER_ID","data":{"ccn":"4242424242424242","exp":"11/30","cvv":"123"}}'
curl -s -X POST -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"update","order-id":"$ORDER_ID","items":[{"id":"1013":10}]}'
faisal@faisal:~$
```

Attribute name	Value	Type	Remove
userId - Sort key	a468b478-f001-70c2-b028-b26664637ed0	String	
address		Map	Remove
confirmationToken	dGZoQNbrqR1	String	Remove
itemList		Map	Remove
1013	10	Number	Remove
orderStatus	210	Number	Remove
paymentTS	1776179114	Number	Remove
totalAmount	330	Number	Remove

After applying the fix, I repeated the same race-condition test by sending billing and update requests concurrently.

Post-fix result

The system returned:

- update: "cart updated"

- billing: "amount": 330

In DynamoDB:

- itemList = 10
- totalAmount = 330

What changed

Previously, it was possible to pay for a smaller quantity while receiving a larger one due to a race condition. After the fix, billing now reflects the correct item quantity, and no mismatch occurs between the order and the charged amount.

Confirmation

The vulnerability no longer succeeds because inconsistent states cannot be created. At the same time, normal behavior still works correctly:

- users can update the cart before payment
- billing processes valid orders normally

Part 9:

1. Intended Logic and Security Rule(s)

The intended behavior of the application is to enforce a strict order-processing workflow. A user is allowed to modify the contents of an order only while the order is in the open (100) state. Once the user initiates the billing process, the order should transition into a non-editable state (such as processing (200)), preventing any further modifications. The security rule is that order data must remain consistent throughout the billing process. This means that once billing begins, the quantity of items and total amount must not

change. The system should ensure that the total charged to the user accurately reflects the final state of the order.

2. Evidence Sources and Behavior Trace

The analysis is based on observations from API requests, terminal outputs, and DynamoDB records.

Normal behavior (expected):

- The user creates or updates an order while it is in the open state.
- The user initiates billing.
- The system calculates the total amount based on the current item list.
- The payment is processed and the order is updated to a paid or processing state.
- The total amount stored in the database matches the items in the order.

Exploit behavior (before fix):

- The attacker sends a billing request and an update request at nearly the same time.
- The billing request calculates the total using the original item quantity.
- The update request modifies the order quantity before the billing process completes.
- The database ends up with inconsistent data, for example:
 - itemList = 10
 - totalAmount = 25

This shows that the user was able to pay for fewer items while receiving more, demonstrating a race condition.

Post-fix behavior:

- The same concurrent requests are executed again.
- The system either locks the order early or ensures the billing calculation uses the updated data.
- The database now shows consistent values, for example:

- itemList = 10
- totalAmount = 330

This confirms that the system no longer allows inconsistent states.

3. Deviation Analysis and Classification

The exploit represents a clear deviation from the intended application logic.

The intended rule is that once billing starts, the order must not be modified. However, before the fix, the system allowed the order to be updated during the billing process. This resulted in a mismatch between the item quantity and the amount charged.

From a security perspective, this is a serious issue because it allows users to manipulate business logic to gain financial advantage. The system fails to enforce proper workflow sequencing, leading to inconsistent and incorrect outcomes.

This vulnerability is best classified as:

- **Intentional misuse / security-relevant abuse**
because an attacker can deliberately exploit timing to bypass the intended logic and obtain more items than paid for.

4. Explainable Fix and Post-Fix Validation

The fix was implemented by enforcing strict workflow control and adding database-level protections.

First, the update function was modified to allow changes only when the order is in the **open (100)** state. If the order status is anything else, the update request is rejected.

Second, the billing function was modified to **lock the order immediately** by changing its status from 100 (open) to 200

(processing) before performing any billing operations. This prevents concurrent update requests from modifying the order.

Third, **DynamoDB conditional expressions** were added to both update and billing operations. These ensure that updates only occur if the order is in the expected state, even under concurrent execution.

These changes address the root cause of the vulnerability by:

- enforcing proper sequencing of operations,
- preventing simultaneous conflicting updates,
- and ensuring data consistency at the database level.

Post-fix validation:

- The race condition test was repeated using concurrent billing and update requests.
- The system no longer produces inconsistent states.
- The total amount always matches the item quantity.
- Legitimate behavior is preserved:
 - users can still update orders before billing,
 - billing works normally for valid requests.

This confirms that the fix successfully restores the intended behavior and eliminates the vulnerability.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Race Condition (Lesson #8)	Orders can only be modified while in open (100) state. Once billing starts, the order must not be changed and the total	API requests (curl), Lambda code (update_order.py, order_billing.py), DynamoDB records	Updating cart before billing works correctly and total amount matches the item list during	Concurrent billing and update requests allowed modifying item quantity after billing started, resulting in mismatch

	must match the item list.		normal checkout	(e.g., itemList = 10, totalAmount = 25)
--	---------------------------	--	-----------------	---

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Logic Race Condition (Lesson #8)	The system allowed order updates during billing, violating the rule that order data must remain consistent once payment starts. This enabled underpayment for more items.	Intentional Misuse / Security-Relevant Abuse	Added validation and conditional updates in update_order.py and early locking (orderStatus = 200) in order_billing.py using DynamoDB ConditionExpression	Repeating the exploit shows no mismatch; total amount always matches item quantity (e.g., itemList = 10, totalAmount = 330)	Before: concurrent requests caused inconsistent state. After: system enforces order locking and consistent results regardless of timing

Part 10:

This vulnerability was caused by the incorrect assumption that requests would be processed in a safe, sequential order. In reality, serverless systems handle requests concurrently, which allowed billing and update operations to overlap and create inconsistent order states.

This issue is especially important in a serverless environment because AWS Lambda functions run independently and in parallel, making race conditions more likely if proper state management is not enforced.

The key lesson is that business logic must be enforced on the server side with strict workflow controls. Using techniques such as state validation, early locking, and

conditional database updates ensures that operations occur in the correct order and prevents concurrent requests from causing security issues.